BHV_Waypoint

Waypoint behavior.

Included MIT MOOS-IvP PDF sources:

- bhv_waypoint.pdf

The Waypoint Behavior

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/bhvdocs/bhv_waypoint`

Contents

1 The Waypoint Behavior

The Waypoint behavior is used for transiting to a set of specified waypoint in the x-y plane. The primary parameter is the set of waypoints. Other key parameters are the inner and outer radius around each waypoint that determine what it means to have met the conditions for moving on to the next waypoint. Them basic idea is shown in Figure 1.

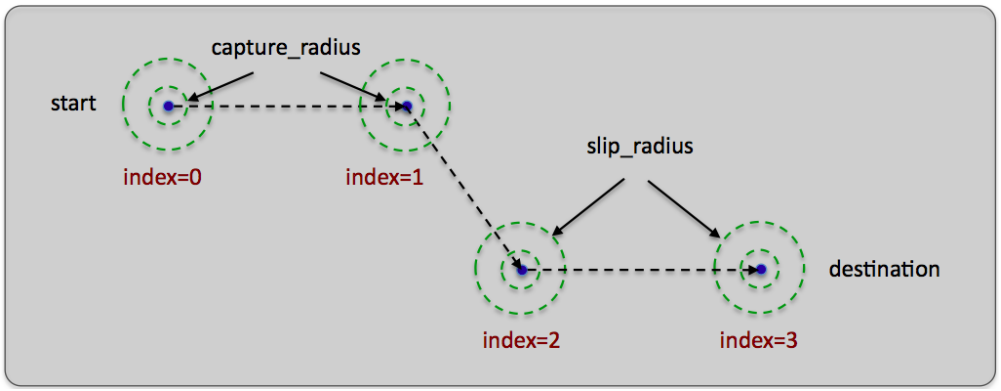

Figure 1: **The Waypoint behavior:** The waypoint behavior basic purpose is to traverse a set of waypoints. A capture radius is specified to define what it means to have achieved a waypoint, and a non-monotonic radius is specified to define what it means to be "close enough" should progress toward the waypoint be noted to degrade.

The behavior may also be configured to perform a degree of track-line following, that is, steering the vehicle not necessarily toward the next waypoint, but to a point on the line between the previous and next waypoint. This is to ensure the vehicle stays closer to this line in the face of external forces such as wind or current. The behavior may also be set to "repeat" the set of waypoints indefinitely, or a fixed number of times. The waypoints may be specified either directly at start-up, or supplied dynamically during operation of the vehicle. There are also a number of accepted geometry patterns that may be given in lieu of specific waypoints, such as polygons, lawnmower pattern and so on.

1.1 Configuration Parameters

Listing 1.1: Configuration Parameters Common to All Behaviors.

`activeflag`:	A MOOS variable-value pair posted when the behavior is in the *active* state. [more].
`condition`:	Specifies a condition that must be met for the behavior to be running. [more].
`duration`:	Time in behavior will remain running before declaring completion. [more].
`duration_idle_decay`:	When true, duration clock is running even when in the *idle* state. [more].
`duration_reset`:	A variable-pair such as `MY_RESET=true`, that will trigger a duration reset. [more].
`duration_status`:	The name of a MOOS variable to which the vehicle duration status is published. [more].
`endflag`:	A MOOS variable-value pair posted when the behavior has completed. [more].
`idleflag`:	A MOOS variable-value pair posted when the behavior is in the *idle* state. [more].
`inactiveflag`:	A MOOS variable-value posted when the behavior is *not* in the *active* state. [more].

name:	The (unique) name of the behavior. [more].
nostarve:	Allows a behavior to assert a maximum staleness for a MOOS variable. [more].
perpetual:	If true allows the behavior to to run even after it has completed. [more].
post_mapping:	Re-direct behavior output normally to one MOOS variable to another instead. [more].
priority:	The priority weight of the behavior. [more].
pwt:	Same as priority.
runflag:	A MOOS variable and a value posted when a behavior has met its conditions. [more].
spawnflag:	A MOOS variable and a value posted when a behavior is spawned. [more].
spawnxflag:	A MOOS variable and a value posted when a behavior is spawned. [more].
templating:	Turns a behavior into a template for spawning behaviors dynamically. [more].
updates:	A MOOS variable from which behavior parameter updates are read dynamically. [more].

Listing 1.2: Configuration Parameters for the Waypoint Behavior.

Parameter	Description
capture_radius:	The radius tolerance, in meters, for satisfying the arrival at a waypoint. The default is 3. Section 1.5.
capture_line:	If set to true, waypoint arrival will be achieved if the vehicle crosses the line perpendicular to the line approaching the waypoint. Default is false. Section 1.6.
crs_spd_zaic_ratio:	Specifies the relative weight of the course portion of the ZAIC. Valid range is $[1, 99]$. Default is 50.
cycleflag:	Optional MOOS variable-value pairs posted at end of each cycle through waypoints.
efficiency_measure:	Determines if leg by leg efficiency is to be measured and/or reported. The default is `"off"`.
lead:	If this parameter is set, track-line following between waypoints is enabled. Section 1.7.
lead_damper:	Distance from trackline within which the `lead` distance is stretched out. Section 1.7.
lead_to_start:	Boolean indicating whether trackline following is applied to first waypoint. The default is false. Section 1.7.
order:	The order in which waypoints are traversed - `"normal"`, `"reverse"`, and with 14.3 or later, `"toggle"`. Section 1.1.
points:	A colon separated list of x,y pairs given as points in 2D space, in meters. Section 1.1.
point:	A single x,y pair given as a point in 2D space, in meters. Section 1.1.
polygon:	An alias for points. Need not be a convex polygon. Section 1.1.
post_suffix:	A suffix tagged onto the WPT_STATUS, WPT_INDEX and CYCLE_INDEX variables.
radius:	An alias for capture_radius. Section 1.5.

repeat: The number of *extra* times traversed through the waypoints. Or `"forever"`. The default is `"normal"` meaning the points will be visited in the order listed. Section 1.4.

slip_radius: An "outer" capture radius. Arrival declared when the vehicle is in this range and the distance to the next waypoint begins to increase. The default is 15 meters. Section 1.5.

speed: The desired speed (m/s) at which the vehicle travels through the points. Accepts only non-negative numbers. The default is 0.

speed_alt: An alternative desired speed (m/s) at which the vehicle travels through the points. Applies only when the use_alt_speed parameter is set to true. The default is -1, which indicates internally that it has not been set. (Introduced after release 15.5.)

use_alt_speed: If true then attempt to use the altnernate speed set with the speed_alt parameter, if that speed is set to a non-negative value. The default is false. (Introduced after release 15.5.)

visual_hints: Optional hints for visual properties in variables posted intended for rendering. Section 1.12.

wpt_status_var: Optional MOOS variable for posting the waypoint status messages as described in Section 1.9.

wpt_index_var: Optional MOOS variable for posting the waypoint index messages as described in Section 1.9.

wptflag: Optional MOOS variable-value pairs posted after each waypoint has been reached.

Listing 1.3: Example Configuration Block.

```
Behavior = BHV_Waypoint
{
  // General Behavior Parameters
  // ---------------------------
  name         = transit               // example
  pwt          = 100                   // default
  condition    = MODE==TRANSITING      // example
  updates      = TRANSIT_UPDATES       // example

  // Parameters specific to this behavior
  // ------------------------------------
     capture_radius = 3                     // default
       capture_line = false                 // default
          cycleflag = COMMS_NEEDED=true     // example
               lead = -1                    // default
       lead_damper = -1                     // default
     lead_to_start = false                  // default
              order = normal                // default
             points = pts={-200,-200:30,-60} // example
        post_suffix = HENRY                 // example
             repeat = 0                     // default
       slip_radius = 15                     // default
             speed = 1.2                    // default is zero
            wptflag = HITPTS = $(X),$(Y)    // example

       visual_hints = vertex_size  = 3            // default
       visual_hints = edge_size    = 1            // default
       visual_hints = vertex_color = dodger_blue  // default
       visual_hints = edge_color   = white        // default
       visual_hints = nextpt_color = yellow       // default
       visual_hints = nextpt_lcolor = aqua        // default
       visual_hints = nextpt_vertex_size = 5      // default
}
```

1.2 Variables Published

The below MOOS variables will be published by the behavior during normal operation, in addition to any configured flags. A variable published by any behavior may be supressed or changed to a different variable name using the post_mapping configuration parameter described in Section **??**.

- WPT_STAT: A comma-separated string showing the status in hitting the list of points.
- WPT_INDEX: The index of the current waypoint. First point has index 0.
- CYCLE_INDEX: The number of times the full set of points has been traversed, if repeating.
- VIEW_POINT: A visual cue for indicating the waypoint currently heading toward.
- VIEW_POINT: A visual cue for indicating the steering point, if the `lead` parameter is used.
- VIEW_SEGLIST: A visual cue for rendering the full set of waypoints.

The Waypoint behavior will also publish any MOOS variables configured with the general behavior flags:

- runflag, idleflag, activeflag, inactiveflag, and endflag, described in Section **??**.

as well as the flags defined locally for the Waypoint behavior:

- `cyleflag` and `wptflag`.

1.3 Specifying Waypoints with the `points` or `point` Parameter

There are a few formats supported for setting the list of waypoints. In the simplest case, when there is just a single point, as with the waypoint return behavior in the alpha mission, the following format may be used:

```
point = 0,0
```

Using the plural `points` parameter is also ok when there is only one point. Another common method is to specify a colon-separated list of comma-separate pairs. For example, also from the alpha mission:

```
points = 60,-40:60,-160:150,-160:180,-100:150,-40
```

Other formats supported are tied to the `XYSegList` and `XYPolyogn` classes. The former represents a set of vertices connected by line segments with a clear first and last vertex. The latter represents a convex polygon over vertices with the notion of the first or last vertex less clear in some cases. So for example, both of the following ways of setting waypoints are supported:

```
points = pts={60,-40:60,-160:150,-160:180,-100:150,-40}
points = format=lawnmower, x=0, y=40, height=60, width=180, lane_width=15
```

When loading a waypoint specification, the behavior will first attempt to build an XYSeglist from one of the above formats. Failing this, it will try to build an XYPolygon from the specification. Therefore any of the XYPologon string formats described in the Geometry documentation is supported. For example:

```
points = format=radial, label=foxtrot, x=0, y=40, radius=60, pts=6, snap=1
points = format=ellipse, label=golf, x=0, y=40, degs=45, pts=14, snap=1,
            major=100, minor=70
```

In specifying a list of points indirectly via the lawnmower, radial or ellipse patterns, the ”first” point in the list may be less predictable.

1.4 The `order` and `repeat` Parameters

The order of the parameters may be reversed with the `order` parameter, and the number of times the waypoints are traversed may be adjusted with the `repeat` parameter. An example specification:

```
points  = 60,-40:60,-160:150,-160:180,-100:150,-40
order   = reverse  // default is "normal"
repeat  = 3        // default is 0
```

A waypoint behavior with this specification will traverse the five points in reverse order (150, -40 first) four times (one initial cycle and then repeated three times) before completing. If there is a syntactic error in this specification at helm start-up, an output error will be generated and the helm will launch in the MALCONFIG mode. If the syntactic error is passed as part of a dynamic update (see Section **??**), the change in waypoints will be ignored and the a warning posted to the BHV_WARNING variable. See the Geometry documentation for more methods for specifying sets of waypoints. The behavior can be set to repeat its waypoints indefinitely by setting repeat="forever". The point parameter may be used insted of points when there is only a single waypoint.

1.5 The capture_radius and slip_radius Parameters

The capture_radius parameter specifies the distance to a given waypoint the vehicle must be before it is considered to have arrived at or achieved that waypoint. It is the inner radius around the points in Figure 1. The slip_radius parameter specifies an alternative criteria for achieving a waypoint.

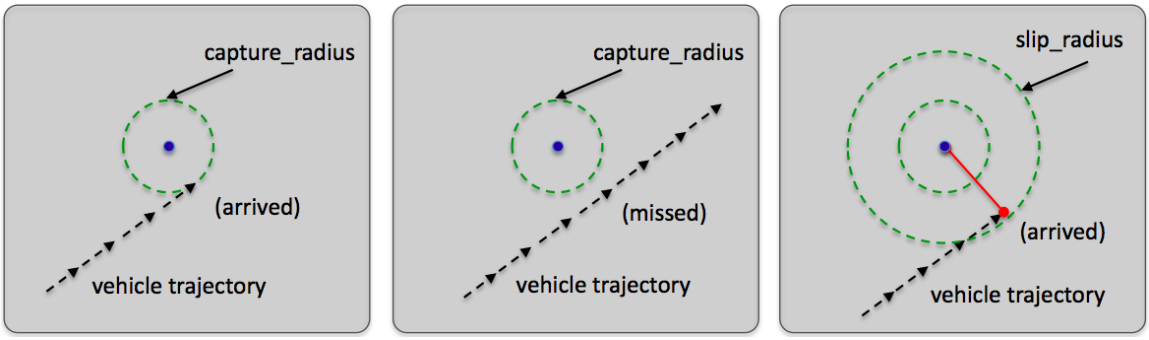

Figure 2: **The capture radius and slip radius:** (a) a successful waypoint arrival by achieving proximity less than the capture radius. (b) a missed waypoint likely resulting in the vehicle looping back to try again. (c) a missed waypoint but arrival declared anyway when the distance to the waypoint begins to increase and the vehicle is within the slip radius.

As the vehicle progresses toward a waypoint, the sequence of measured distances to the waypoint decreases monotonically. The sequence becomes non-monotonic when it hits its waypoint or when there is a near-miss of the waypoint capture radius. The slip_radius, is a capture radius distance within which a detection of increasing distances to the waypoint is interpreted as a waypoint arrival. This distance would have to be larger than the capture radius to have any effect. As a rule of thumb, a distance of twice the capture radius is practical. The idea is shown in Figure 2. The behavior keeps a running tally of hits achieved with the capture radius and those achieved with the slip radius. These tallies are reported in a status message described in Section 1.9 below.

1.6 The capture_line Parameter

The capture_line parameter allows for an alternative or additional arrival criteria to be applied. When set to true, waypoint arrival will be achieved if the vehicle crosses the line perpendicular to the line segment approaching the waypoint from the previous waypoint. In the case of the first

waypoint, the "previous" waypoint is defined by the vehicle position when the behavior first becomes active.

When capture_line is set to true, both the capture line criteria *and* the capture and slip radius criteria apply. If either is satisfied, then arrival is achieved. If the capture_line criteria is set instead to absolute, then the capture and slip radius criteria is disabled, and only the capture line criteria is achieved.

When capture_line is set to false, only the capture and slip radius criteria are applied. A note of caution - setting the capture_line parameter to absolute essentially sets the capture and slip radius to zero. If the capture_line parameter is subsequently set to anything else (true or false), the capture and slip radius values also need to be explicitly re-set to non-zero values.

1.7 Track-line Following using the lead, lead_damper, and lead_to_start Parameters

By default the waypoint behavior will output a preference for the heading that is directly toward the next waypoint. By setting the lead parameter, the behavior will instead output a preference for the heading that keeps the vehicle closer to the track-line, or the line between the previous waypoint and the waypoint currently being driven to.

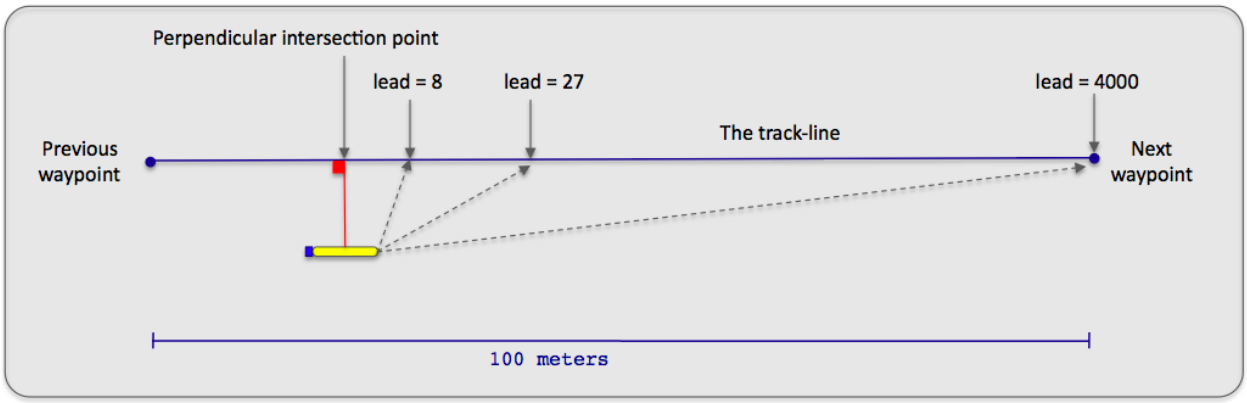

Figure 3: **The track-line mode:** When in track line mode, the vehicle steers toward a point o the track line rather than simply toward the next waypoint. The steering-point is determined by the lead parameter. This is the distance from the perpendicular intersection point toward the next waypoint.

The distance specified by the lead parameter is based on the perpendicular intersection point on the track-line. This is the point that would make a perpendicular line to the track-line if the other point determining the perpendicular line were the current position of the vehicle. The distance specified by the lead parameter is the distance from the perpendicular intersection point toward the next waypoint, and defines an imaginary point on the track-line. The behavior outputs a heading preference based on this imaginary steering point. If the lead distance is greater than the distance to the next waypoint along the track-line, the imaginary steering point is simply the next waypoint.

Normally, when trackline following is enabled, it is enabled only between the first waypoint and all succeess waypoints. When the parameter lead_to_start is set to true, trackline following is

attempted even to the first waypoint, by defining a trackline from the vehicle's present position when the behavior first enters the running mode.

If the lead parameter is enabled, it may be optionally used in conjuction with the lead_damper parameter. This parameter expresses a distance from the trackline in meters. When the vehicle is within this distance, the value of the lead parameter is stretched out toward the next waypoint to soften, or dampen, the approach to the trackline and reduce overshooting the trackline.

1.8 The wptflag Parameter

The wptflag parameter allows the user to speicify flags to be posted upon the achievement of each waypoint, where waypoint achievement is defined by the normal means, depending on how the user has configured the capture_radius, slip_radius or capture_line parameters. The wptflags are in the same format as idle, end, active, or inactive flags defined generally for IvP behaviors, i.e, they consist of a MOOS variable and value. For example:

```
wptflag = STATION_KEEP=true
```

The flags also support a handleful of macro expansions that allow the certain information about ownship or the next waypoint to be embedded in the posting that may not be available until run-time. These macros are:

- $[X] or $(X): Expanded to ownship's x position in local coordinates.
- $[Y] or $(Y): Expanded to ownship's x position in local coordinates.
- $[NX] or $(NX): Expanded to the x position of the next waypoint in local coordinates.
- $[NY] or $(NY): Expanded to the y position of the next waypoint in local coordinates.

Here are a couple examples of how the wptflag parameter may be useful in practice.

In the first case, in the example provided above, the mission may be configured to station-keep after arriving at each waypoint. This may be useful for a surface vehicle with a primary mission of collecting sensor measurements at periodic points in a survey area corresponding to waypoints. In this case, another station-keeping behavior is activated with the wptflag posting, which presumably temporarily idles the waypoint behavior while measurements are collected.

A second case utilizes the wptflag parameter to post the x-y position of the next waypoint in the list.

1.9 Variables Published by the Waypoint Behavior

The waypoint behavior publishes five variables for monitoring the performance of the behavior as it progresses: WPT_STAT, WPT_INDEX, CYCLE_INDEX, VIEW_POINT, VIEW_SEGLIST. The WPT_STAT contains information identifying the vehicle, the index of the current waypoint, the type of hits recorded for each waypoint, the distance to the current waypoint, and the estimated time of arrival to the current waypoint. Example output:

```
WPT_STAT = "vname=alpha,behavior=traverse1,index=0,hits=10/11,dist=43,eta=23"
```

The "hits=10/11" component in the above example indicates that, of the 11 waypoint arrivals achieved so far, 10 of them were achieved by meeting the capture radius criteria, and one of them was achieved by meeting the nonmonotonic radius criteria.

The WPT_INDEX variable simply publishes the index of the current waypoint. This is a bit redundant since this information is also in the WPT_STAT posting, but this variable is logged as a numerical variable, not a string, and facilitates the plotting of the index value as a step function in post mission analysis tools. The CYCLE_INDEX variable publishes the number of times the behavior has traversed the entire set of waypoints. The behavior may be configured to post the information in these three variables using alternative variables of the user's liking:

```
post_mapping  = WPT_STAT, MY_WPT_STATUS_VAR
post_mapping  = WPT_INDEX, MY_WPT_INDEX_VAR
post_mapping  = CYCLE_INDEX, MY_CYCLE_INDEX
```

or, to suppress the reports completely:

```
post_mapping  = WPT_STAT, SILENT
post_mapping  = WPT_INDEX, SILENT
post_mapping  = CYCLE_INDEX, SILENT
```

Further posts to the MOOSDB can be configured to be made at the end of each cycle, that is, after reaching the last waypoint. Normally, if the repeat parameter remains at its default value of zero, then the end of a cycle and completing are identical and endflags can be used to post the desired information. However, when the behavior is configured to repeat the set of waypoints one or more times before completed, the cycleflag parameter may be used to post one or more variable-value pairs at the end of each cycle. Likewise, if the repeat parameter is zero, but the behavior is set with perpetual=true, the cycle flags will posted each new time that the behavior completes.

The VIEW_POINT and VIEW_SEGLIST variables provide information consumable by a GUI application such as pMarineViewer or alogview for rendering the set of waypoints traversed by the behavior (VIEW_SEGLIST) and the behavior's next waypoint (VIEW_POINT). These two variables are responsible for the visual output in the Alpha Example Mission in Section **??** in Figure **??**.

1.10 The Objective Function Produced by the Waypoint Behavior

The waypoint behavior produces a new objective function, at each iteration, over the variables `speed` and `course/heading`. The behavior can be configured to generate this objective function in one of two forms, either by coupling two independent one-variable functions, or by generating a single coupled function directly. The functions rendered in Figure 4 are built in the first manner.

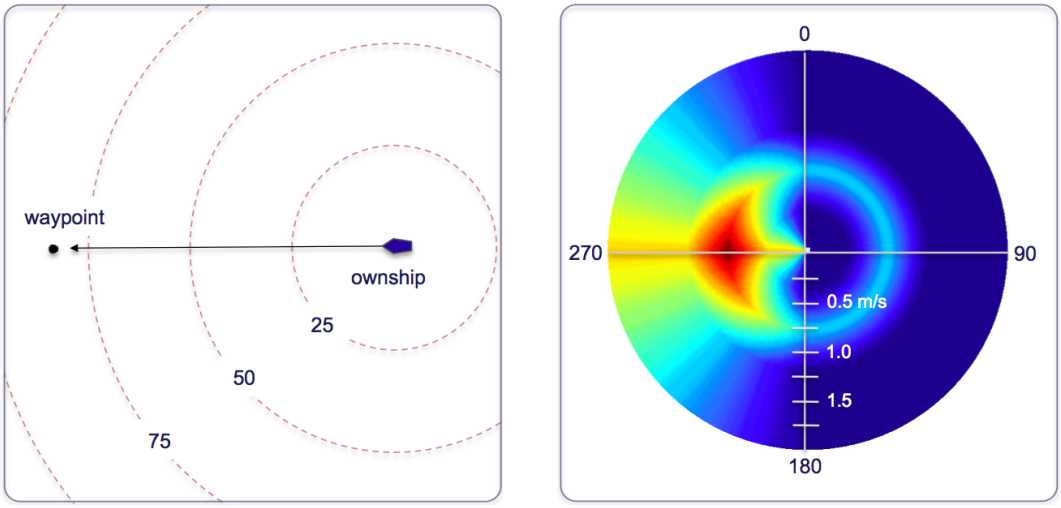

Figure 4: **A waypoint objective function:** The objective function produced by the waypoint behavior is defined over possible heading and speed values. Depicted here is an objective function favoring maneuvers to a waypoint 270 degrees from the current vehicle position and favoring speeds closer to the mid-range of capable vehicle speeds. Higher speeds are represented farther radially out from the center.

1.11 Further Clarification on the `repeat` vs. `perpetual` Parameter

It's worth clarifying the difference in usage and effect between the `repeat` parameter, which is specific to the Waypoint behavior, and the `perpetual` parameter which is defined for all helm behaviors. Normally when a behavior completes, it is entered into the completed stated, never again to be called upon by the helm. See Section **??** for more on behavior run states. It is up to the behavior implementor to decide what it means to be complete, and the implentor typically invokes the `setComplete()` function from within the code. See Section **??** for more on this function. In the case of the Waypoint behavior, completion by default occurs when the vehicle has hit all its waypoints. By setting `perpetual=true` the behavior, upon hitting all its waypoints, still invokes the `setComplete()` function, which causes it to post its endflags, but it does not enter the completed state. This feature is used, for example, in the Alpha example mission to allow the behavior to repeatedly return to the start point and be re-deployed, and later return to its start point again using the same Waypoint behavior.

The `repeat` parameter is used to change the criteria for completion. Whereas the normal criteria for completion is hitting all waypoints once, using `repeat=N` changes the criteria to be hitting all waypoints `N+1` times. A few rules of thumb may be helpful in keeping things straight:

- When `perpetual=false`, the default setting, the behavior will permanently enter the completed state once it has hit all its waypoints.
- When `perpetual=true` and the behavior does not post endflags leading to its run conditions being unsatisfied, the behavior will repeat its waypoints indefinitely.
- When `perpetual=true` and it does indeed post endflags that lead to its run conditions being unsatisfied, it will remain in a running state until all its waypoints are hit. If the `repeat` parameter is used, it won't post its endflags until it has repeated all waypoints the specified

number of times. During the course of traversal, the cycleflag posts will be made each time it has completed the set of waypoints. Upon completion, it will post its endflags and reset its cycle counter.

- By setting repeat=N, where N>0, the perpetual parameter is automatically set to true.

1.12 Visual Hints Defined for the Waypoint Behavior

Although the primary output of the Waypoint behavior is an IvP Function, a number of visual properties are also published for convenience in mission monitoring. This includes (a) the set of waypoints, (b) the point the behavior is presently progressing toward, and (c) the trackpoint, if trackpoint following is enabled. These visual artifacts have default properties in size and color that may be altered to the user's preferences. These preferences are configurable through the visual_hints parameter. Each parameter below is used in the following way by example:

```
visual_hints = vertex_size=3, edge_size=2
visual_hints = vertex_color=khaki
```

- vertex_size: The size of vertices rendered in the loiter polygon. The default is 1.
- edge_size: The width of edges rendered in the loiter polygon. The default is 1.
- vertex_color: The color of vertices rendered in the loiter polygon. The default is "dodger_blue".
- edge_color: The color of edges rendered in the loiter polygon. The default is "white".
- label_color: The color of label for set of waypoints. The default is "white".
- nextpt_color: The color of the point rendered as the present next waypoint. The default is "yellow".
- nextpt_lcolor: The color of the label for the point rendered as the present next waypoint. The default is "aqua".
- nextpt_vertex_size: The size of the vertex for the point rendered as the present next waypoint. The default is 1.

Rendering of vertices or the next waypoint may be shut off with a size of zero, and labels may be shut off with the special color "invisible". For a list of legal colors, see Appendix **??**.